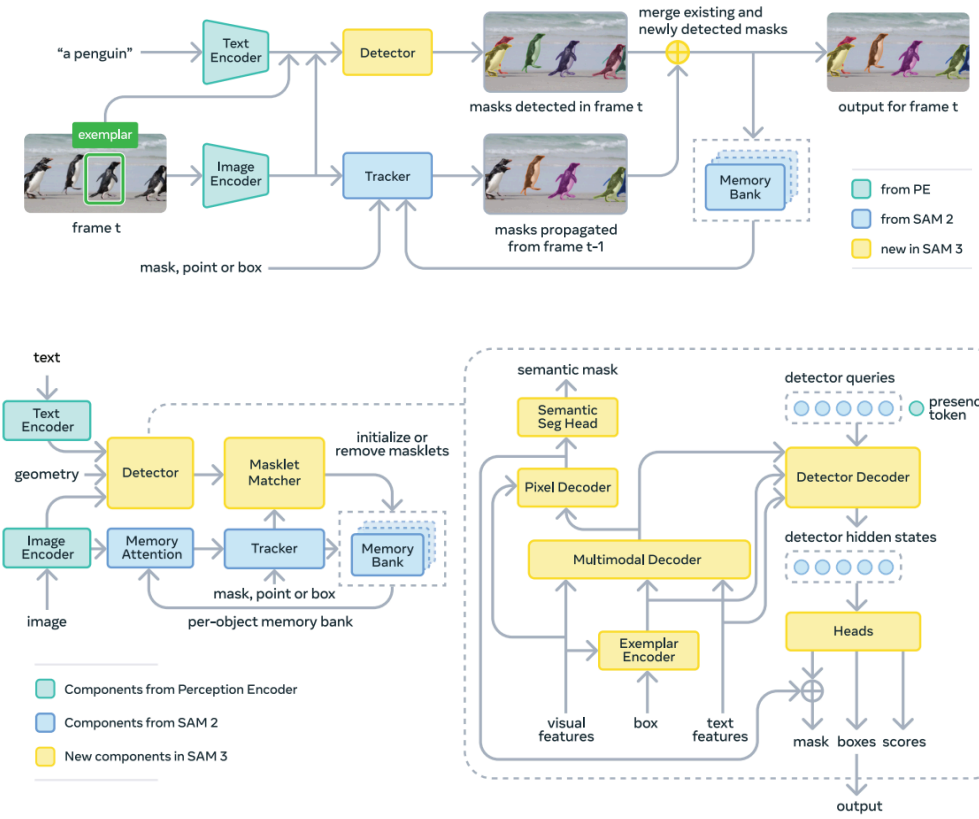
SAM3

▼

1. Architecture (부록 내용 일부만 들어간 버전)



▼ 1.1 Detector Architecture (탐지기 구조)

전반적인 아키텍처는 DETR 패러다임을 따르고 있다.

- 이미지와 텍스트 프롬프트는 PE(Perception Encoder)에서 인코딩
- 이미지 견본(프롬프트)는 존재할 때만 견본 인코더에서 인코딩
- 이미지 견본 토큰과 텍스트 토큰을 합쳐서 "프롬프트 토큰"
- 이미지 인코더의 조건화되지 않은 임베딩이 퓨전 인코더에 입력
- 퓨전 인코더는 입력받은 임베딩을 프롬프트 토큰과 cross attention하여 조건화

- 퓨전 인코더는 DETR-like 디코더와 연결

- DETR-like 디코더는 퓨전 인코더에서 조건화된 임베딩을 학습된 객체 쿼리와 cross attention
- 디코더 레이어는 각 객체 쿼리에 대한 분류 로짓(프롬프트에 해당하는지 안 하는지 이진 라벨링) + 이전 디코더 레이어의 예측 박스에 대해 정확한 박스로 가기 위한 필요 변화량 델타 출력 (기타에서 표 참고)
- 각 객체에 어텐션이 집중되도록 박스 영역 위치 편향을 사용(box-region-positional bias)
- 최근 DETR 모델들과 달리 초기 바닐라 어텐션 연산을 응용 (기타에서 상단 3개의 표 아래쪽 줄글 참고)

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B_{\text{box}}\right)V$$

[학습]

- 이중 지도 학습 : 바닐라 어텐션 기반 학습의 비효율(셀프 어텐션, 크로스 어텐션이 길항 작용을 하는데 번갈아 배치되어 있음) 해결
- 학습 시 동일한 객체 쿼리와 가중치를 공유하는 2개의 디코더 경로를 병렬로 두고 각각 다른 방식으로 학습

  1. **기본 디코더 (`O-Decoder`):** 셀프 어텐션과 크로스 어텐션을 모두 포함한 표준 디코더.
  정답 객체 1개당 쿼리 1개만 매칭하는 일대일 매칭으로 학습 → 중복X

  2. **보조 디코더 (`C-Decoder`):** 셀프 어텐션X. 크로스 어텐션 + FFN만 남긴 디코더.
  중복 제거 기능X → 정답 객체 1개에 여러 개의 쿼리를 양성으로 할당하는 일대다 매칭으로 학습 ⇒ 크로스 어텐션이 객체 주변으로 쿼리를 모으고 특징을 추출하는 데만 집중하도록 만듭니다.

실제 추론할 때는 보조 디코더(`C-Decoder`)를 버리고 기본 디코더(`O-Decoder`)만 사용

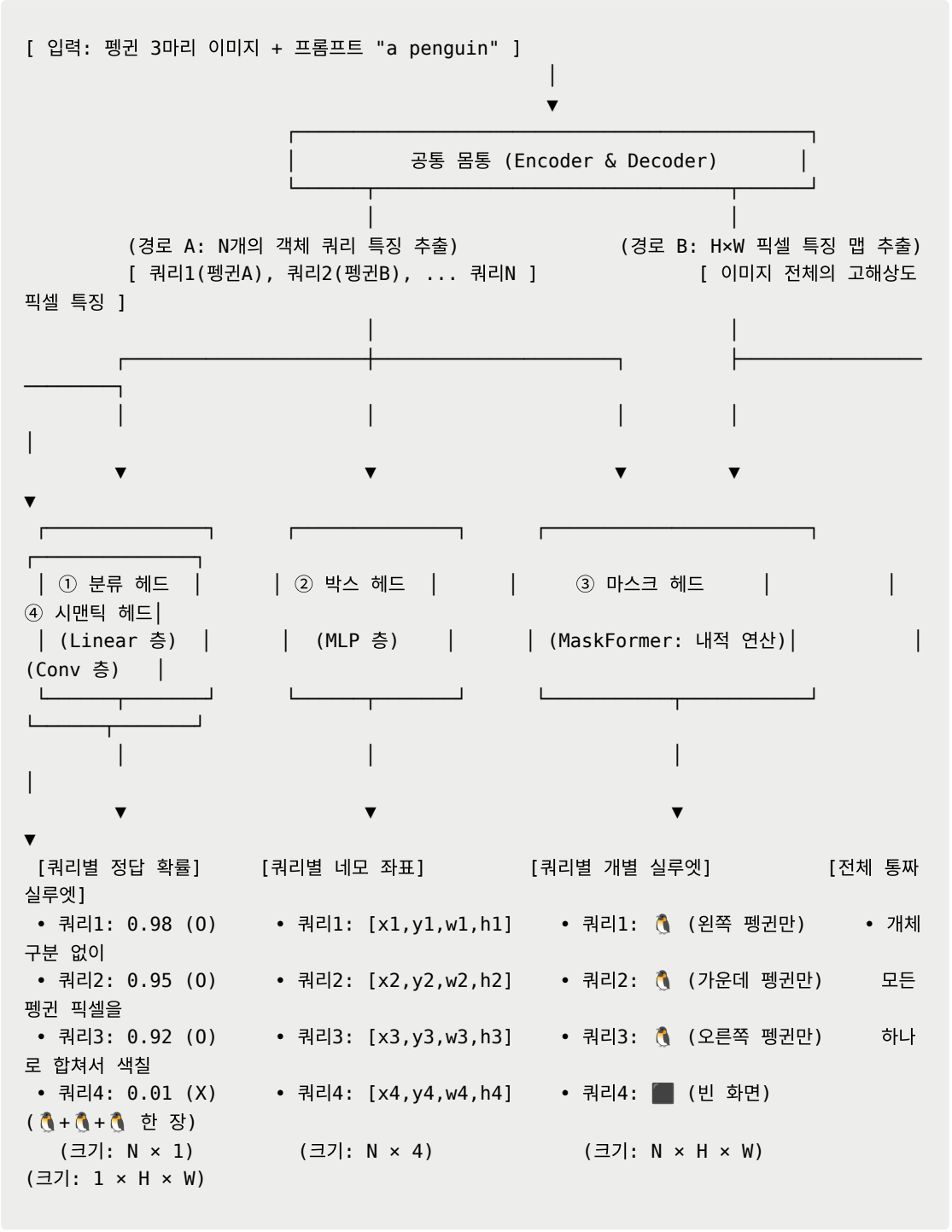
- Align 손실함수 이용 : 분류 점수, 박스 위치에 대한 평가 점수를 같이 계산

$$t = s^{\alpha} \cdot u^{1-\alpha}$$

- 박스를 부정확하게 친 쿼리는 절대 높은 분류 점수를 낼 수 없고, 박스를 정확하게 친 쿼리만 높은 분류 점수를 낼 수 있음.

- 마스킹 헤드 + 시멘틱 세그멘테이션 헤드

- 마스킹 헤드 : 펭귄이 3마리 있으면 펭귄 1, 펭귄 2, 펭귄 3을 각각 별개의 마스크로 분리
  +) “1번 쿼리가 찾는 특징과 닮은 픽셀만 불을 켜라!”라고 해서 1번 펭귄 모양만 남기는 방식
- 시멘틱 세그멘테이션 헤드 : 몇 번 펭귄인지는 몰라도, 펭귄에 해당하는 모든 픽셀은 전부 1이 되도록 픽셀 특징을 잘 만들라고 가이드



▼ 기타 (내가 이해 못해서 공부한 것들)

구분	박스 (Bounding Box)	마스크 (Mask)
형태	객체를 둘러싸는 **직사각형 테두리**	객체의 실제 모양을 따라 픽셀 단위로 칠한 **영역(실루엣)**
데이터 표현	**4개의 좌표 숫자** ($b = \{b_x, b_y, b_w, b_h\} \in [0, 1]^4$) *(중심점 x, y 및 너비 w, 높이 h)*	**2차원 픽셀 행렬** ($H \times W$ 크기의 이진 맵) *(각 픽셀이 객체에 속하면 1, 아니면 0 또는 확률/로짓)*
배경 포함 여부	사각형 안에 객체뿐만 아니라 **주변 배경도 함께 포함됨**	배경을 제외하고 **객체에 해당하는 픽셀만 정확히 분리함**
주요 태스크	객체 탐지 (Object Detection)	이미지 분할 (Instance / Semantic Segmentation)

구분	분류 로짓 (classification logit)	마스크 (mask)
예측 대상	**객체 쿼리(object query) 단위**	**이미지 픽셀(pixel) 단위**
의미 (이진 라벨의 뜻)	"이 쿼리가 포착한 객체가 프롬프트 (예: `"a penguin"`)에 **해당하는 객체인가, 아닌가?**"	"이 픽셀이 해당 객체의 **몸체(영역)에 속하는가, 배경인가?**"
데이터 크기 (객체 1개당)	**숫자 1개** (스칼라 값)	$H \times W$**개의 숫자** (2차원 픽셀 맵)
역할	찾은 후보가 프롬프트가 요구한 대상이 맞는지 **정답 여부(존재·분류 신뢰도)를 채점**	객체의 **윤곽선과 형태를 픽셀 단위로 색칠**

출력 종류	예측 단위	객체 쿼리 1개당 크기	전체 출력 텐서 크기 (N: 객체 쿼리 수)
분류 로짓 (`classification logit`)	객체 쿼리별 이진 여부	숫자 1개	$N \times 1$
바운딩 박스 델타 (`delta`)	객체 쿼리별 좌표·크기 변화량	숫자 4개 ($\Delta x, \Delta y, \Delta w, \Delta h$)	$N \times 4$
인스턴스 마스크 (`mask head`)	객체 쿼리별 픽셀 영역	$H \times W$	$H \times W \times N$
시맨틱 마스크 (`semantic`	이미지 전체 픽셀별 이진 여부	*(쿼리 무관)*	$H \times W \times 1$

`segmentation head`)			

- **초기 DETR :** 디코더의 각 객체 쿼리(object query)가 이미지의 모든 픽셀(특징 맵 전체)과 표준 크로스 어텐션을 수행. 학습 초기, 어텐션 가중치가 이미지 전체에 균일 분포 → 특정 객체의 위치로 초점을 좁히기까지 학습 수렴 ↓
- **최근 DETR 계열 모델 (Deformable DETR):** 이 문제를 해결하기 위해 표준 어텐션이 아니라, 기준점이나 박스 주변의 **소수 샘플링 좌표만 부분적으로 참조하는 변형 어텐션(Deformable Attention)** 등 특수하게 설계된 로컬 어텐션 연산으로 대체
- **마스크, 시멘틱 헤드 둘 다 쓰는 이유:** 마스크 헤드는 디코더의 '객체 쿼리'와 이미지의 '픽셀 특징 맵'을 곱(내적)해서 마스크를 만듭니다(MaskFormer Figure 2 참조). 이때 시맨틱 헤드를 픽셀 특징 맵에 직접 붙여 학습시키면, 픽셀 특징 맵 자체가 프롬프트("펭귄") 영역을 훨씬 더 선명하게 구분하게 되어 결과적으로 마스크 헤드의 성능까지 함께 올라갑니다.

▼ 1.2 Presence Token (존재성 토큰)

모든 각각의 객체 쿼리에 대해 모든 프레임과 이미지에서 recognition & location을 하는 것은 쉽지 않다.

⇒ 하나의 이미지 내, 전체에서의 문맥적 신호가 중요 → 전역 문맥을 이해하도록 하는 것은 비생산적임.

⇒ Recognition, Localization 작업을 분리 by 전역적 존재성 토큰 (Global Presence Token)

이 토큰 하나만으로 해당 이미지/프레임 내에 NP(명사구)의 형태인 타겟 개념이 존재하는지를 예측 → P(NP is present in input) : 확률

각 객체 쿼리 q_i는 Localization 문제만 해결하면 된다. P(q_i is matched | NP is present in input)
⇒ NP가 이미지/프레임 내에 존재할 때 위치가 제대로 맞는 확률

[각 객체 쿼리의 최종 스코어]

1차 객체 쿼리 스코어 × 존재성 스코어(Presensce Score)

▼

1.3 Image Exemplars and Interactivity (이미지 견본과 상호작용성)

이미지 견본 : 바운딩 박스 + 양성/음성 이진 라벨
⇒ 명확한 분리?구분? 또는 텍스트 프롬프트를 보충하는 용도

⇒ 프롬프트에 해당하는 모든 개체 탐지

[SAM1·2 vs SAM3]

1. 입력 : 개체에 박스 (개)
2. 탐지 :
    a. SAM1·2 : 해당 개체만 탐지·추적 (처음 표시했던 개 한 마리)
    b. SAM3 : 해당 개체가 속한 클래스에 포함되는 모든 개체 탐지·추적 (모든 개에 개1, 개2, 개3, ...)
3. 출력 :
    a. SAM1·2 : 대상 개체(개 한 마리)에 마스크 + 라벨(개)
    b. SAM3 : 대상 개체들(개1, 개2, 개3, ...)에 모두 마스크 + 라벨(개1, 개2, 개3, ...)

[각각의 이미지 견본은 구분되어 인코딩] ⇒ Concatenation ⇒ Transformer(소형)

▼ 인코딩은 다음을 포함

- 포지셔널 임베딩 : 좌표
- 라벨 임베딩 : 양성/음성
- 관심 영역 풀링 시각 특징 (ROI-pooled visual features) : 박스 안에 어떻게 생긴 객체가 들어있는가

⇒ 텍스트 프롬프트와 Concatenate ⇒ 프롬프트 토큰 구성

[상호작용성]

1. 1차 탐지 결과 확인

2. 오류 기반 피드백
    a. 미탐 ⇒ 미탐 개체에 양성 박스 추가
    b. 오탐 ⇒ 오탐 개체에 음성 박스 추가
3. 출력 결과 수정

▼

1.4 Tracker and Video Architecture (추적기와 영상 구조)

영상 V, 프롬프트 P ⇒ Detector ⇒ Tracker

- 각 프레임에서 탐지기가 개체 O_t 탐지
- 추적기가 이전 프레임에서 시공간 마스크 M_{t-1} 전파 →현재 프레임에서의 시공간 마스크 $\hat{M}_t$
- 전파된 마스크 $\hat{M}_t$와 새로운 개체 O_t간의 매칭 함수

$$\hat{M}_t = Propagate(M_{t-1})$$
$$O_t = Detect(I_t, P)$$
$$M_t = MatchAndUpdate(\hat{M}_t, O_t)$$

▼

1.5 Tracking an Object with SAM 2 Style Propagation

첫 프레임에서는 전달받을 M_{t-1}가 없음 → Detector가 프롬프트에 해당하는 개체를 모두 찾음

⇒ 3개의 개체를 탐지했다면 '추적 기록부'에 마스크릿1, 마스크릿2, 마스크릿3 등을 초기값으로 등록

⇒ 두번째 프레임부터는 이전 프레임에서의 마스크릿들을 바탕으로 추적기가 새로운 마스크릿의 위치$\hat{M}_t$를 예측하며 추적

[추적기]

- 추적기는 탐지기와 같이 같은 PE 백본의 이미지/프레임 인코더 공유
- 탐지기 학습 후, PE를 unlearnable로 전환 → 추적기는 SAM2에서와 같이 학습

- 프롬프트 인코더, 마스크 디코더, 메모리 인코더, 메모리 뱅크
- 현재 프레임의 시각 특징을 과거 프레임들의 기억(Memory)으로 조건화
- **메모리 뱅크 :** 과거 프레임, 기조건화(기준) 프레임에서 추출한 특징으로 객체의 생김새 저장
- **메모리 인코더** : 현재 프레임의 이미지 특징에 과거의 기억 결합
  - 현재 프레임에서 시각적 특징 간의 셀프 어텐션
  - 현재 프레임에서 시각적 특징과 메모리 뱅크의 공간 특징으로 크로스 어텐션 수행
    - Q : 현재 프레임의 시각 특징
    - K/V : 메모리 뱅크의 공간 특징 + 기조건화(기준) 프레임의 공간 특징
    - 현재 프레임의 각 영역 Q가 메모리 뱅크에 저장된 과거 객체의 생김새 및 위치와 얼마나 일치하는지

[메모리 저장]

추론 시에는 객체가 높은 신뢰도로 존재하는 프레임들만 메모리 뱅크에 저장

▼ WHY?

비디오에서 객체를 추적할 때는 객체가 다른 물체 뒤에 가려지거나 화면 밖으로 잠시 벗어나거나, 유사하게 생긴 다른 객체와 겹치는 상황이 빈번하게 발생합니다. 만약 모든 프레임의 예측 결과를 무조건 메모리 뱅크에 저장하면 다음과 같은 치명적인 문제가 발생합니다.

- 자기회귀적 오류 누적
- 메모리 오염과 드리프트 : 이미 오염된 메모리를 참조하느라 엉뚱한 배경이나 다른 물체를 추적하게 되는 **드리프트(drift) 현상**이 발생

▼ HOW?

1. Mask Decoder의 Occlusion Head와 IoU Prediction Head 활용
2. Detector를 활용한 주기적 재프롬프팅으로 메모리 신뢰도 보강

(자세한 내용은 뒤에)

[마스크 디코더] - 아직 잘 이해 못했음

인코더의 은닉 상태와 출력 토큰 사이의 양방향 트랜스포머

모호성을 해결하기 위해 신뢰도와 함께 마스크 3개 예측 → 앙상블

▼ 기타

```
[현재 프레임의 시각 특징]
        │
        ├── ① Self-Attention  : 현재 프레임 내부의 장면·맥락 파악
        │
        └── ② Cross-Attention : 메모리 뱅크(과거·기준 프레임)에서 추적 대상 객체의 특
징·위치 정보 가져오기
        │
        ▼
[기억이 반영된 현재 프레임 특징 (다음 문단의 encoder hidden states)] ──► 마스크 디코더
(Mask Decoder)로 전달하여 현재 프레임 마스크 예측
```

- 크로스 어텐션에서의 K/V : 픽셀 단위의 공간 메모리 특징과 객체 수준의 추상 정보를 담은 객체 포인터로 구성
- 크로스 어텐션 연산 목적 : 현재 프레임의 객체 쿼리 위치(Q)가 과거 프레임들에서 특정 객체가 있었던 위치 및 외형 특징(K, V)과 어텐션 연산을 수행함으로써, "과거에 추적하던 그 객체가 현재 프레임 t의 어디로 이동했는가?"를 반영한 특징 맵 출력
  ⇒ 이 출력이 Mask Decoder로 들어가 현재 프레임의 마스크 $\hat{M}_t$를 예측
- **Memory Encoder (메모리 쓰기 및 생성):** 현재 프레임에서 마스크 예측이 끝난 후, 예측된 마스크와 PE 백본의 조건화되지 않은 토큰을 융합하여 새로운 공간 메모리를 만들어 Memory Bank에 저장
- PE 백본은 프레임당 단 한 번만 실행되어 조건화되지 않은 프레임 임베딩을 생성하고, 이를 두 개의 인코더(퓨전 인코더, 메모리 인코더)로 동시에 전송
- Detector의 Fusion Encoder가 '프롬프트 토큰'에 Cross-Attention하여 개념을 찾는 것과 대칭적으로, Tracker는 'Memory Bank의 과거 공간 메모리'에 Cross-Attention하여 특정 객체의 ID를 추적
- **Tracker는 Detector가 조건화해 놓은 특징을 이어받지 않음.**
  ⇒ PE 백본에서 출력된 '아무 조건도 부여되지 않은 순수 시각 특징'을 Detector와 Tracker가 각자 받아서, **서로 다른 대상(프롬프트 vs. 과거 메모리)으로 각자 조건화**

구분	Detector 경로: Fusion Encoder	Tracker 경로: Memory Attention (본문의 Memory Encoder)

공통 입력 (Q)	PE 백본의 Unconditioned Frame Embeddings	PE 백본의 Unconditioned Frame Embeddings
구조	Self-Attention + Cross-Attention 트랜스포머	Self-Attention + Cross-Attention 트랜스포머
Cross-Attention 대상 (K, V)	**Prompt Tokens** (텍스트 토큰 + 이미지 예시 토큰)	**Memory Bank** (과거 프레임의 Spatial Memories + Object Pointers)
조건화 목적	**의미론적 검출 (Identity-agnostic):** 현재 프레임에서 프롬프트 개념(예: "펭귄")에 해당하는 모든 영역 활성화	**개체별 추적 (Identity-preserving):** 현재 프레임에서 특정 ID의 과거 객체(예: "1번 펭귄")가 이동한 영역 활성화

- 객체 포인터

구분	공간 메모리 (Spatial Memory)	객체 포인터 (Object Pointer)
텐서 형태	2D 특징 맵 격자 ($H' \times W' \times C$)	단일(또는 소수) $1D$ 벡터 ($1 \times D$)
생성 모듈	**Memory Encoder:** 현재 프레임의 예측 마스크와 PE 백본의 비조건화 특징을 합성하여 생성	**Mask Decoder:** 마스크를 생성할 때 사용된 디코더의 출력 토큰에서 직접 추출하여 생성
담고 있는 정보	**국소적·기하학적 정보 :** 객체의 세밀한 경계선, 위치, 텍스처 등 픽셀 단위의 공간 정보	**전역적·의미론적 정보 :** 공간 좌표가 제거된, 해당 객체 자체의 고차원 외형 및 정체성 요약 정보

▼ 1.6 Matching and Updating Based on Detections

추적된 마스크 $\hat{\mathcal{M}}_t$을 얻은 후, 간단한 IoU 기반 매칭 함수를 통해 현재 프레임의 탐지 결과 $\mathcal{O}_t$와 매칭하고, 그것들을 현재 프레임의 마스크릿 목록인 $\mathcal{M}_t$에 추가(등록)

1. IoU 기반 : 추적기가 쫓아온 마스크($\hat{\mathcal{M}}_t$)와 탐지기가 현재 프레임에서 찾은 마스크($\mathcal{O}_t$)가 화면상에서 얼마나 겹치는지 비교하여 같은 객체끼리 매칭(매핑)
2. 마스크릿 목록 $\mathcal{M}_t$에 추가 : 매칭이 완료된 마스크들을 마스크릿 목록에 추가
3. 남은 탐지 객체 처리 : 탐지기가 현재 프레임에서 찾은 마스크($\mathcal{O}_t$)가 기존에 추적해온 마스크($\hat{\mathcal{M}}_t$)와 겹치는 부분이 없으면 새로운 마스크릿 번호 할당 후 마스크릿 목록에 추가

$$\mathcal{M}_t = \underbrace{\text{Validated_and_Updated}(\hat{\mathcal{M}}_t)}_{\text{기존 추적 객체 등록 (”add them to } \mathcal{M}_t\text{”)}} \cup \underbrace{\text{Spawn}(\mathcal{O}_t \setminus \mathcal{O}_t^{\text{matched}})}_{\text{매칭 안 된 신규 검출 객체 등록}}$$

⇒ 중복 생성 방지 + 오류 마스크 추적 방지

Masklet = Mask(공간) + Tracklet(시간) → 마스크릿은 마스크의 시퀀스 집합
→예 : Masklet 1 = $\{\mathcal{M}_1^1, \mathcal{M}_2^1, \ldots, \mathcal{M}_t^1\}$
→ 1번 마스크릿 : 1번 개체 마스크(프레임 내 특정 위치)가 1~t 프레임(시간축)에 있음

[매핑과 반영 구체]

1. **IoU 매칭을 통한 매칭 점수(MDS) 갱신 :**
   - 기존 마스크릿 i의 예측 마스크 $\hat{\mathcal{M}}_t^i$와 현재 프레임 검출 객체 $d \in \mathcal{O}_t$ 사이의 IoU를 계산합니다.
   - $\mathrm{IoU}(d, \hat{\mathcal{M}}_t^i) > \mathrm{iou_threshold}$(기본값 0.1)를 만족하는 검출 객체 d가 하나라도 있으면 해당 프레임에서 매칭 성공($\Delta_i(t) = +1$), 없으면 매칭 실패($\Delta_i(t) = -1$)로 판정합니다.
   - 이를 시간 구간 동안 합산한 점수인 MDS(Masklet Detection Score, $S_i = \sum \Delta_i(\tau)$)를 업데이트합니다.
2. **점수 미달 마스크릿의 필터링 :**
   - 만약 특정 마스크릿의 누적 점수 S_i가 임계값 아래로 떨어지면(즉, Tracker 혼자만 쫓아가고 있고 Detector는 그 위치에서 객체를 전혀 검출하지 못하면), 해당 마스크릿을 아예 삭제하거나 마스크를 0으로 만들어(Suppression) 현재 프레임 출력 $\mathcal{M}_t$에 나타나지 않게 억제합니다.
     - 마스크릿을 삭제하기 :
       - 어떤 물체가 처음 발견되어 마스크릿이 생성된 직후 **초기 15프레임** 동안, 탐지기와의 매칭 점수 합계(S_i)가 0보다 작을 때
       - 오탐으로 판단하여, **추적기(Tracker)의 내부 상태와 메모리에서 삭제**
       - 두 마스크릿이 같은 물체를 중복 추적할 때 나중에 생긴 마스크릿을 삭제하는 규칙도 있음
     - 마스크를 0으로 만들기 :
       - 초기 수습 기간을 통과했거나 추적 중인 마스크릿인데, 처음 등장한 시점(t^i_{first})부터 현재 프레임(τ)까지의 **전체 누적 점수** $S_i(t^i_{\mathrm{first}}, \tau)$

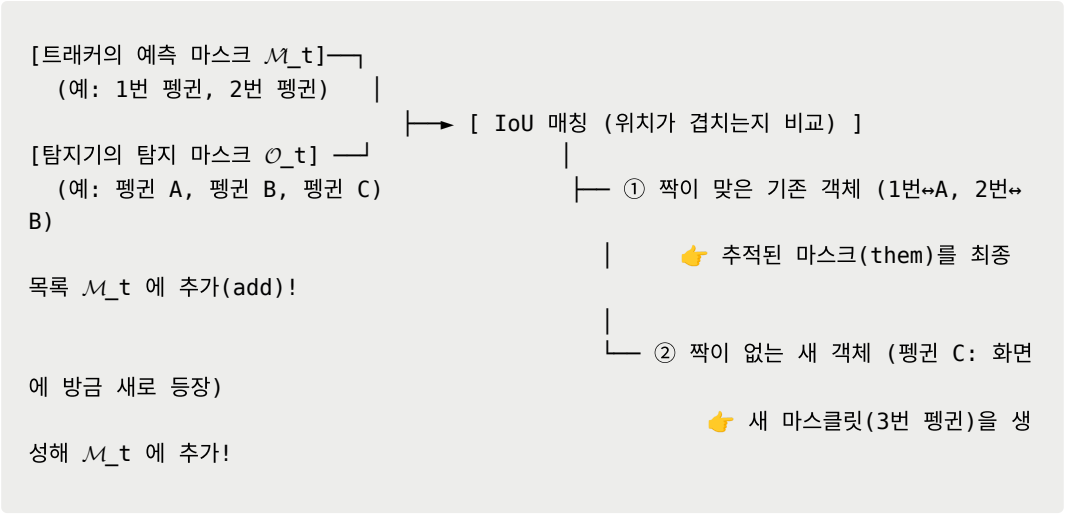
가 0 미만(음수)으로 떨어졌을 때입니다.

- 추적기 내부(메모리 뱅크 등)에서는 이 객체를 삭제하지 않고 뒤에서 몰래 계속 추적 상태를 유지하되, 이번 프레임에서 화면에 내보내는 $H \times W$ 마스크 행렬의 모든 픽셀 값을 0으로 출력

3. **마스크 교체 여부의 결정 :**
   - 정상적으로 매칭되어 살아남은 마스크릿 i를 $\mathcal{M}_t$에 넣을 때, 기본적인 프레임에서는 Tracker가 전파한 마스크 $\hat{\mathcal{M}}_t^i$를 그대로 $\mathcal{M}_t$에 넣습니다.
   - 재프롬프팅 조건(매 $N = 16$번째 프레임에서 $\text{IoU} \geq 0.8$이고 양쪽 신뢰도가 0.8을 넘거나, 반대로 Tracker 마스크가 번져서 매칭된 검출 객체와의 박스 IoU가 0.85 미만으로 떨어졌을 때)에 해당하면, Tracker의 예측 마스크 $\hat{\mathcal{M}}_t^i$ 대신 **매칭된 Detector의 마스크 $d \in \mathcal{O}_t$로 값을 덮어씌워 (replace)** $\mathcal{M}_t$와 Memory Bank에 넣습니다.
     - **탐지기의 신뢰도 :** Presence 점수 $\times$ 객체 쿼리 점수
     - **추적기의 신뢰도 :** 추적기의 Mask Decoder가 3개의 후보 마스크를 만들면서 함께 출력하는 자체 품질 예상 점수(predicted IoU confidence)

▼ 기타 정리

기호	본문 명칭	누가 만든 결과인가?	의미
$\hat{\mathcal{M}}_t$	tracked masks	**트래커 (Tracker)**	이전 프레임의 객체($\mathcal{M}_{t-1}$)가 현재 프레임(t)에서 어디로 이동했을지 **트래커가 예측한 후보 마스크들** (^는 예측값을 뜻함)
$\mathcal{O}_t$	current frame detections	**탐지기 (Detector)**	과거 기억 없이 오직 현재 프레임(t) 이미지 1장만 보고 **탐지기가 새로 찾아낸 객체 마스크들**
$\mathcal{M}_t$	(현재 프레임 최종 마스클릿)	**매칭 및 업데이트 모듈**	$\hat{\mathcal{M}}_t$과 $\mathcal{O}_t$를 대조한 뒤, **현재 프레임(t)의 최종 결과로 확정 지어 담는 '최종 마스크릿 목록(바구니)'**



▼ 1.7 Instance Refinement with Visual Prompts

양성/음성 클릭을 통해 사용자의 즉각적인 프롬프트 수정

양성 클릭 : 점 추가

음성 클릭 : 점 제거